~\.bob\playground\ad_attack_lab\kerberoast_lab.py

```
"""
Active Directory Attack Lab - Kerberoasting & GPO Hardening
============================================================
Author  : Security Research Lab
Purpose : Demonstrate Kerberoasting attack simulation and GPO-based defenses
          in a virtual Windows domain environment.

DISCLAIMER: For authorized lab/research use only. Do NOT use against
            production or unauthorized systems.
"""

import os
import sys
import time
import hashlib
import argparse
import logging
import subprocess
from datetime import datetime
from typing import Optional

# ---------------------------------------------------------------------------
# Logging Setup
# ---------------------------------------------------------------------------
logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s] %(levelname)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("AD-Lab")


# ---------------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------------
LAB_BANNER = r"""
╔══════════════════════════════════════════════════════════╗
║        Active Directory Attack Lab - Research Tool       ║
║         Kerberoasting Simulation & GPO Hardening         ║
╚══════════════════════════════════════════════════════════╝
  [!] Authorized Lab Use Only
"""

# Simulated SPN accounts (lab environment only)
LAB_SPN_ACCOUNTS = [
    {"account": "svc_sql",     "spn": "MSSQLSvc/dc01.lab.local:1433",  "ticket_type":
"RC4_HMAC"},
```

```
    {"account": "svc_web",     "spn": "HTTP/webserver.lab.local:80",    "ticket_type": "AES256"},
    {"account": "svc_backup",  "spn": "BAK/backupsrv.lab.local:445",   "ticket_type": "RC4_HMAC"},
    {"account": "svc_monitor", "spn": "MON/monitor.lab.local:8080",    "ticket_type": "AES256"},
]

# GPO Hardening baseline policies
GPO_POLICIES = {
    "PasswordPolicy": {
        "MinimumPasswordLength": 14,
        "PasswordComplexity": True,
        "MaxPasswordAge": 60,
        "MinPasswordAge": 1,
        "PasswordHistorySize": 24,
    },
    "AccountLockoutPolicy": {
        "LockoutThreshold": 5,
        "LockoutDuration": 30,
        "ObservationWindow": 30,
    },
    "KerberosPolicy": {
        "MaxTicketAge": 10,
        "MaxRenewAge": 7,
        "MaxServiceAge": 600,
        "MaxClockSkew": 5,
        "TicketValidateClient": True,
    },
    "AuditPolicy": {
        "AuditLogonEvents": "Success,Failure",
        "AuditAccountLogon": "Success,Failure",
        "AuditPrivilegeUse": "Failure",
        "AuditObjectAccess": "Success,Failure",
        "AuditPolicyChange": "Success",
        "AuditAccountManagement": "Success,Failure",
    },
    "ServiceAccountControls": {
        "EnforceAES256Only": True,
        "DisableRC4": True,
        "RequireManagedServiceAccounts": True,
        "LimitSPNRegistration": True,
    },
}


# ---------------------------------------------------------------------------
# Helper Utilities
# ---------------------------------------------------------------------------

def banner() -> None:
```

```
    """Print the lab banner."""
    print(LAB_BANNER)


def simulate_tgt_request(username: str, domain: str) -> dict:
    """
    Simulate requesting a Ticket Granting Ticket (TGT) from the KDC.
    In a real lab this would use impacket's AS-REQ flow.
    """
    logger.info(f"[AS-REQ] Requesting TGT for {username}@{domain}")
    time.sleep(0.3)
    fake_tgt = {
        "client":    f"{username}@{domain}",
        "server":    f"krbtgt/{domain}",
        "issued":    datetime.utcnow().isoformat(),
        "ticket_id": hashlib.md5(f"{username}{domain}{time.time()}".encode()).hexdigest(),
    }
    logger.info(f"[AS-REP] TGT issued: {fake_tgt['ticket_id'][:16]}...")
    return fake_tgt


def simulate_tgs_request(tgt: dict, spn: str, ticket_type: str) -> dict:
    """
    Simulate requesting a TGS (service ticket) for a given SPN.
    In a real lab this uses the TGT to obtain an encrypted service ticket.
    """
    logger.info(f"[TGS-REQ] Requesting service ticket for SPN: {spn}")
    time.sleep(0.2)
    raw_hash = hashlib.sha256(f"{spn}{tgt['ticket_id']}{ticket_type}".encode()).hexdigest()
    tgs = {
        "spn":         spn,
        "ticket_type": ticket_type,
        "tgs_hash":    f"$krb5tgs$23${raw_hash[:32]}${'a' * 32}${raw_hash[32:]}",
        "issued":      datetime.utcnow().isoformat(),
    }
    logger.info(f"[TGS-REP] Ticket type {ticket_type} received for {spn}")
    return tgs


def assess_ticket_crackability(ticket: dict) -> dict:
    """
    Assess the offline crackability risk of a harvested service ticket.
    RC4_HMAC tickets are significantly weaker than AES256.
    """
    risk_map = {
        "RC4_HMAC": {
            "risk":   "HIGH",
            "reason": "RC4_HMAC (etype 23) is fast to crack offline with tools like
hashcat.",
            "recommendation": "Upgrade service account to AES256 and disable RC4 via GPO.",
```

```
        },
        "AES256": {
            "risk":   "LOW",
            "reason": "AES256-CTS-HMAC-SHA1-96 (etype 18) is computationally expensive to crack.",
            "recommendation": "Continue enforcing AES256. Use Managed Service Accounts (MSA/gMSA).",
        },
    }
    assessment = risk_map.get(ticket["ticket_type"], {"risk": "UNKNOWN", "reason": "", "recommendation": ""})
    assessment["spn"] = ticket["spn"]
    return assessment


# ---------------------------------------------------------------------------
# Phase 1 - SPN Enumeration
# ---------------------------------------------------------------------------

def enumerate_spns(domain: str, simulate: bool = True) -> list:
    """
    Enumerate Service Principal Names in the domain.
    In simulation mode uses the built-in lab accounts.
    In live mode (requires domain connectivity) runs: setspn -Q */*
    """
    logger.info(f"[*] Enumerating SPNs in domain: {domain}")
    if simulate:
        logger.info("[*] Simulation mode — using lab SPN dataset")
        for acct in LAB_SPN_ACCOUNTS:
            logger.info(f"    Found SPN: {acct['spn']}  (account: {acct['account']})")
        return LAB_SPN_ACCOUNTS

    # Live mode — only works inside a domain-joined Windows machine
    logger.warning("[!] Live mode: running setspn query (requires domain connectivity)")
    try:
        result = subprocess.run(
            ["setspn", "-Q", "*/*"],
            capture_output=True, text=True, timeout=30
        )
        lines = result.stdout.splitlines()
        spns = []
        for line in lines:
            if "/" in line:
                spns.append({"spn": line.strip(), "ticket_type": "RC4_HMAC", "account": "unknown"})
        logger.info(f"[+] Found {len(spns)} SPNs")
        return spns
    except FileNotFoundError:
        logger.error("setspn not found. Run this on a domain-joined Windows host.")
        return []

```

```python

# ---------------------------------------------------------------------------
# Phase 2 - Kerberoasting Attack
# ---------------------------------------------------------------------------

def kerberoast(domain: str, attacker_user: str, simulate: bool = True) -> list:
    """
    Execute the Kerberoasting attack:
    1. Authenticate to KDC and obtain TGT.
    2. Request TGS for every enumerated SPN.
    3. Harvest the encrypted TGS hashes for offline cracking.
    """
    logger.info("=" * 60)
    logger.info("[PHASE 2] Kerberoasting Attack")
    logger.info("=" * 60)

    spn_accounts = enumerate_spns(domain, simulate=simulate)
    if not spn_accounts:
        logger.error("No SPNs found. Aborting Kerberoasting phase.")
        return []

    tgt = simulate_tgt_request(attacker_user, domain)
    harvested = []

    for entry in spn_accounts:
        tgs = simulate_tgs_request(tgt, entry["spn"], entry["ticket_type"])
        assessment = assess_ticket_crackability(tgs)
        harvested.append({
            "account":    entry["account"],
            "spn":        entry["spn"],
            "ticket":     tgs,
            "assessment": assessment,
        })

    logger.info(f"[+] Harvested {len(harvested)} service ticket(s)")
    return harvested


# ---------------------------------------------------------------------------
# Phase 3 - Hash Output / Simulated Crack
# ---------------------------------------------------------------------------

def display_hashes(harvested: list) -> None:
    """Display harvested TGS hashes and crackability risk."""
    print("\n" + "=" * 60)
    print("  Harvested TGS Hashes (Kerberoast Output)")
    print("=" * 60)
    for item in harvested:
        print(f"\n  Account  : {item['account']}")
        print(f"  SPN      : {item['spn']}")
```

```
        print(f"  Hash     : {item['ticket']['tgs_hash'][:60]}...")
        print(f"  Risk     : {item['assessment']['risk']}")
        print(f"  Reason   : {item['assessment']['reason']}")
        print(f"  Fix      : {item['assessment']['recommendation']}")
    print()


def simulate_hashcat_crack(harvested: list) -> None:
    """
    Simulate what hashcat would do with RC4-based hashes.
    Prints equivalent hashcat command; does NOT perform real cracking.
    """
    print("\n[Simulated Hashcat Commands]")
    print("-" * 60)
    for item in harvested:
        if item["ticket"]["ticket_type"] == "RC4_HMAC":
            print(
                f"  hashcat -m 13100 -a 0 {item['account']}_hash.txt wordlist.txt --rules=best64.rule"
            )
        else:
            print(f"  # {item['account']} uses AES256 — cracking is computationally infeasible")
    print()


# ---------------------------------------------------------------------------
# Phase 4 - GPO Hardening Audit
# ---------------------------------------------------------------------------

def audit_gpo_hardening() -> dict:
    """
    Audit current lab GPO settings against hardening baseline.
    In a real environment this would parse RSoP (Resultant Set of Policy)
    or use PowerShell Get-GPResultantSetOfPolicy.
    """
    logger.info("=" * 60)
    logger.info("[PHASE 4] GPO Hardening Audit")
    logger.info("=" * 60)

    results = {}
    for policy_area, settings in GPO_POLICIES.items():
        logger.info(f"[*] Auditing: {policy_area}")
        area_results = {}
        for key, expected in settings.items():
            # Simulation: assume all checks pass in a hardened lab
            area_results[key] = {"expected": expected, "status": "PASS"}
        results[policy_area] = area_results

    return results
```

```
289
290
291 def print_gpo_report(audit: dict) -> None:
292     """Pretty-print the GPO hardening audit report."""
293     print("\n" + "=" * 60)
294     print("  GPO Hardening Audit Report")
295     print("=" * 60)
296     for area, settings in audit.items():
297         print(f"\n  [{area}]")
298         for key, result in settings.items():
299             status_icon = "✓" if result["status"] == "PASS" else "✗"
300             print(f"    {status_icon} {key}: {result['expected']}  [{result['status']}]")
301     print()
302
303
304 # ---------------------------------------------------------------------------
305 # Phase 5 - Remediation Recommendations
306 # ---------------------------------------------------------------------------
307
308 REMEDIATION_STEPS = [
309     {
310         "id": "REM-01",
311         "title": "Disable RC4 Encryption for Kerberos",
312         "gpo_path": "Computer Configuration → Windows Settings → Security Settings → "
313                     "Local Policies → Security Options",
314         "setting": "Network security: Configure encryption types allowed for Kerberos → "
315                    "AES128_HMAC_SHA1, AES256_HMAC_SHA1 only",
316         "severity": "Critical",
317     },
318     {
319         "id": "REM-02",
320         "title": "Use Group Managed Service Accounts (gMSA)",
321         "gpo_path": "N/A — Implement via PowerShell: New-ADServiceAccount",
322         "setting": "Replace all service accounts with gMSAs (auto-rotating 240-char
    passwords).",
323         "severity": "Critical",
324     },
325     {
326         "id": "REM-03",
327         "title": "Enforce Strong Password Policy for Service Accounts",
328         "gpo_path": "Computer Configuration → Windows Settings → Security Settings → "
329                     "Account Policies → Password Policy",
330         "setting": "Minimum 25 characters, complexity enabled, max age 90 days.",
331         "severity": "High",
332     },
333     {
334         "id": "REM-04",
335         "title": "Enable Kerberos Audit Logging",
336         "gpo_path": "Computer Configuration → Windows Settings → Security Settings → "
337                     "Advanced Audit Policy",
```

```
        "setting": "Audit Kerberos Service Ticket Operations: Success and Failure.",
        "severity": "High",
    },
    {
        "id": "REM-05",
        "title": "Restrict SPN Registration Permissions",
        "gpo_path": "Active Directory Users & Computers → Delegate Control",
        "setting": "Remove write-SPN permission from standard users and service accounts.",
        "severity": "Medium",
    },
    {
        "id": "REM-06",
        "title": "Deploy Microsoft ATA / Defender for Identity",
        "gpo_path": "N/A — Deploy sensor on Domain Controller",
        "setting": "Detects abnormal Kerberos TGS request patterns (Kerberoasting IOC).",
        "severity": "High",
    },
]


def print_remediation_report() -> None:
    """Print remediation recommendations."""
    print("\n" + "=" * 60)
    print("  Remediation Recommendations")
    print("=" * 60)
    for step in REMEDIATION_STEPS:
        print(f"\n  [{step['id']}] {step['title']}")
        print(f"  Severity : {step['severity']}")
        print(f"  GPO Path : {step['gpo_path']}")
        print(f"  Setting  : {step['setting']}")
    print()


# ---------------------------------------------------------------------------
# CLI Entry Point
# ---------------------------------------------------------------------------

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Active Directory Attack Lab — Kerberoasting & GPO Hardening"
    )
    parser.add_argument("--domain",   default="lab.local",   help="Target domain (default:
lab.local)")
    parser.add_argument("--user",     default="attacker",    help="Attacker username
(default: attacker)")
    parser.add_argument("--simulate", action="store_true",   default=True,
                        help="Use simulation mode (default: True)")
    parser.add_argument("--phase",    choices=["all", "enum", "attack", "gpo", "remediate"],
                        default="all", help="Run specific phase (default: all)")
    parser.add_argument("--output",   default=None,          help="Save report to file")
```

```
    return parser.parse_args()


def run_lab(args: argparse.Namespace) -> None:
    banner()
    logger.info(f"Domain   : {args.domain}")
    logger.info(f"User     : {args.user}")
    logger.info(f"Mode     : {'Simulation' if args.simulate else 'Live'}")
    logger.info(f"Phase    : {args.phase}")
    print()

    harvested = []

    if args.phase in ("all", "enum", "attack"):
        harvested = kerberoast(args.domain, args.user, simulate=args.simulate)
        display_hashes(harvested)
        simulate_hashcat_crack(harvested)

    if args.phase in ("all", "gpo"):
        audit = audit_gpo_hardening()
        print_gpo_report(audit)

    if args.phase in ("all", "remediate"):
        print_remediation_report()

    if args.output:
        with open(args.output, "w") as f:
            f.write(f"AD Attack Lab Report — {datetime.utcnow().isoformat()}\n")
            f.write(f"Domain: {args.domain} | User: {args.user}\n\n")
            for item in harvested:
                f.write(f"SPN: {item['spn']} | Risk: {item['assessment']['risk']}\n")
        logger.info(f"[+] Report saved to: {args.output}")


if __name__ == "__main__":
    args = parse_args()
    run_lab(args)

```